

You're reading for free via [Pere Martra's](#) Friend Link. [Upgrade](#) to access the best of Medium.

★ Member-only story

How to Prune LLaMA 3.2 and Similar Large Language Models

This article presents a structured pruning technique for state-of-the-art models, that uses a GLU architecture, enabling the creation of smaller and more efficient large language models.



Pere Martra  · Follow

Published in TDS Archive

14 min read · Nov 27, 2024



Listen



Share

... More

Disclaimer: This article was originally written in Spanish and translated into English using AI tools as support to ensure accuracy and consistency. You can find the original Spanish version [here](#).

As large language models continue to grow in size to achieve greater capabilities, the demand for more efficient, smaller versions has become more necessary than ever. However, reducing a model's size without losing its core functionality is a delicate balancing act.

Techniques such as quantization and pruning are commonly used to decrease size, while methods like knowledge distillation or transfer learning help retain or recover the capabilities lost during the reduction process.

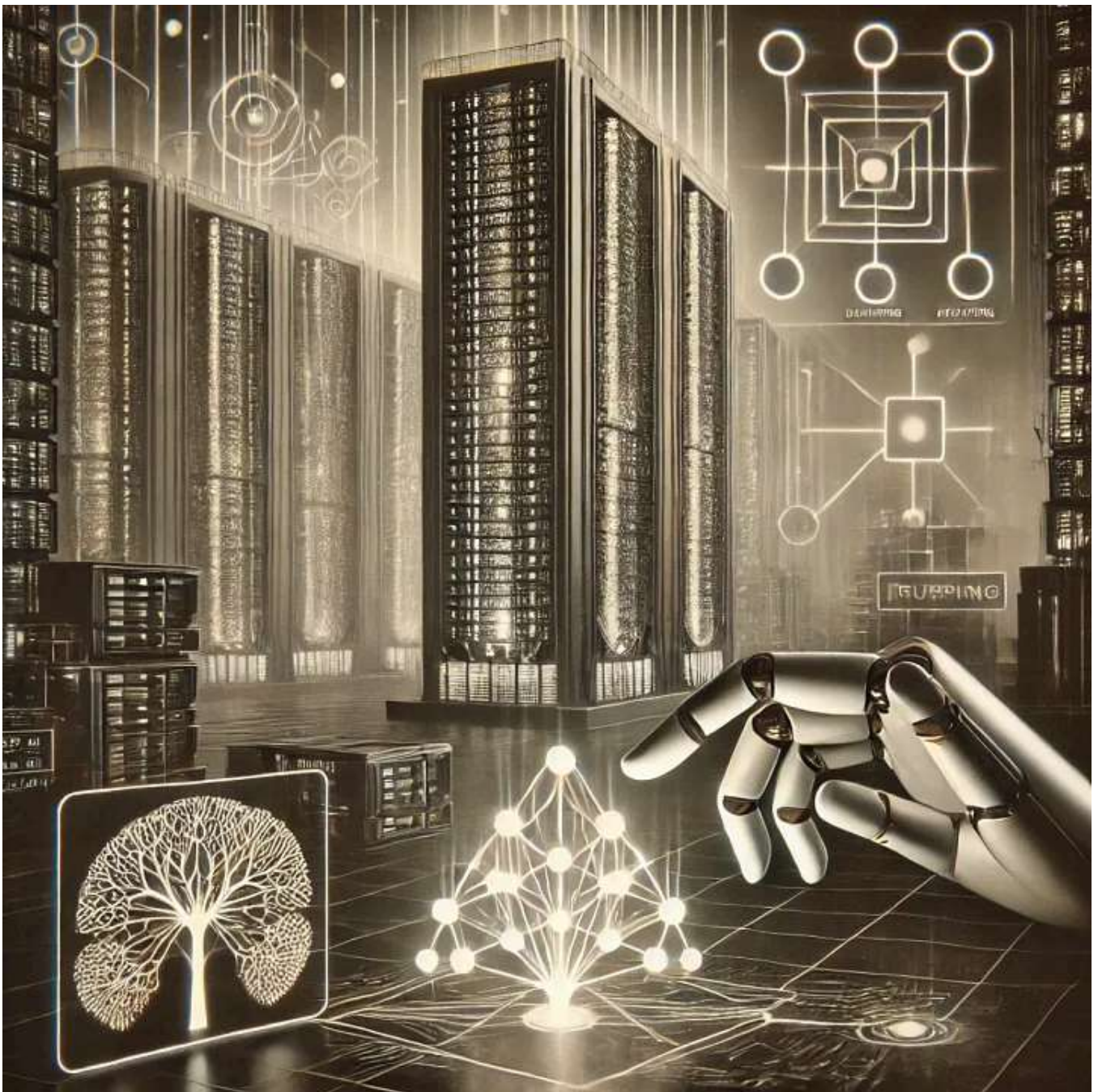


Image generated by author with GPT 4.

Among these, **pruning** stands out as one of the most effective strategies for reducing model size. Unlike quantization, which simplifies numerical representations, pruning involves removing specific parts of the model, such as neurons or entire layers. But this effectiveness comes at a cost: pruning is challenging to apply correctly. Not only do you need to identify which part of the model to prune, but you must also carefully select the elements to remove to minimize the impact on the model's capabilities.

This article focuses on structured width pruning, where selected neurons are removed, and demonstrates how to apply it effectively on MLP layers with a Gated Linear Unit (GLU) structure. By following the steps outlined, you'll see how pruning

can significantly reduce model size while preserving its ability to generate coherent outputs and perform well on key benchmarks.

What is Pruning and how it affects the models?

As I've explained earlier, pruning involves removing parts of the model that are believed to contribute the least to its final output. By carefully selecting these less critical components, pruning aims to create a more efficient model with fewer parameters and reduced computational requirements, without sacrificing its core capabilities.

The primary challenge in pruning lies in deciding which parts of the model to remove. Not all sections of a model impact its performance equally; each serves a distinct purpose.

To illustrate this, let's examine the structure of the model used in this article: LLaMA 3.2-1B.

```
LlamaForCausalLM(  
  (model): LlamaModel(  
    (embed_tokens): Embedding(128256, 2048)  
    (layers): ModuleList(  
      (0-15): 16 x LlamaDecoderLayer(  
        (self_attn): LlamaSdpaAttention(  
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)  
          (k_proj): Linear(in_features=2048, out_features=512, bias=False)  
          (v_proj): Linear(in_features=2048, out_features=512, bias=False)  
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)  
          (rotary_emb): LlamaRotaryEmbedding()  
        )  
        (mlp): LlamaMLP(  
          (gate_proj): Linear(in_features=2048, out_features=8192, bias=False)  
          (up_proj): Linear(in_features=2048, out_features=8192, bias=False)  
          (down_proj): Linear(in_features=8192, out_features=2048, bias=False)  
          (act_fn): SiLU()  
        )  
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)  
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)  
      )  
    )  
    (norm): LlamaRMSNorm((2048,), eps=1e-05)  
    (rotary_emb): LlamaRotaryEmbedding()  
  )  
)
```

```
(lm_head): Linear(in_features=2048, out_features=128256, bias=False)
)
```

When examining the structure, we can identify three main blocks that can be targets for pruning: the embeddings, the self-attention mechanism, and the MLP layers. To decide which of these should be the focus of the pruning process, it's essential to understand the potential benefits and the possible impacts on the model.

The first step is to assess how much each of these sections occupies within the model, giving us an idea of the potential reduction in size.

Parameter Distribution Analysis.

Embeddings and output layer (`embed_tokens`, `lm_head`):

- $128256 \times 2048 \approx 262\text{M}$ parameters per layer
- Two layers totaling 524M parameters

Self-attention mechanism (`self_attn`):

- 16 layers, each containing four projection sub-layers
- Per layer: $2048 \times (2048 + 512 + 512 + 2048) \approx 10.5\text{M}$ parameters
- Total: $10.5 \times 16 \approx 168\text{M}$ parameters

MLP layers (`mlp`):

- 16 layers with GLU structure (`gate_proj`, `up_proj`, and `down_proj`)
- Per layer: $2048 \times 8192 + 2048 \times 8192 + 8192 \times 2048 \approx 50\text{M}$ parameters
- Total: $50 \times 16 \approx 805\text{M}$ parameters

As we can see, the MLP layers represent more than 50% of the model's size, making them clear candidates for pruning. However, before making this decision, it's crucial to understand the contribution of each section to the model's behavior.

Impact Analysis.

The **embedding layers** are responsible for transforming the inputs into dense vector representations that the model can process effectively. Pruning the embedding layer can lead to a loss of the model's ability to understand certain words, or at least reduce the capacity to create vectors that correctly capture the semantic meaning of the inputs. If you want to create a highly specific model that only uses a very specific portion of its input vocabulary, for example, a model for financial or medical analysis, pruning this layer could be an option.

The **attention mechanism** allows the model to focus on the most relevant parts of the input sequence when processing each token. It computes a weighted importance score between every pair of tokens in the input sequence, enabling the model to capture Context and Focus on Relevant Information. Pruning this section can reduce the model's ability to perform tasks requiring a broad understanding of the input context, such as text summarization or translation. It also affects the coherence of generated text.

The **MLP layers** accompany the attention mechanism and enhance the model's ability to understand complex patterns through a series of data expansions and contractions. Pruning this section can limit the model's response to unseen data or tasks not covered during training. In other words, it reduces the model's generalization capability and its ability to provide coherent responses to unfamiliar inputs.

Once you've decided which section of the model to target, the next step is to determine whether to perform width pruning, removing individual neurons, or depth pruning, removing entire layers.

As you can see, pruning a model is quite a complex process that involves making many decisions. You not only have to evaluate the abilities of the resulting model but also its capacity to be trained. These models are designed with the intention of being fine-tuned, usually for specific tasks, so they can be more effective and efficient than the base model for the tasks they are created to perform.

Characteristics of Gated Linear Units

The Gated Linear Unit (GLU) architecture is commonly used in modern neural networks, including LLaMA, Gemma, Mistral, Qwen and similar large language models. GLU introduces an element-wise gating mechanism that allows the model to selectively filter and control the flow of information. This architecture consists of

paired layers, typically: `gate_proj`, `up_proj`, and `down_proj` (as seen in the model structure above), that work together to expand and contract data.

This mechanism enables the model to process more complex patterns while maintaining efficiency. However, it also means that the layers within a GLU structure are tightly coupled, and pruning these layers requires careful consideration.

Any operation on one layer (e.g., removing neurons) must be mirrored in its corresponding paired layers. For instance, if a neuron is removed from `gate_proj`, the same neuron must also be removed from `up_proj`, and the size of the `down_proj` layer must be adjusted accordingly. Most importantly, when calculating the importance of neurons to decide which ones to keep, you need to evaluate the pair of neurons together.

Disrupting the balance of these layers can result in degraded performance or even complete model failure, even if only a small percentage of neurons are removed.

Pruning a Llama 3.2 Model.

The example will be demonstrated using a Llama model, but the code has also been tested successfully with Gemma and QWen models.

You can access the full code in a notebook on my Github Repository.

GitHub - peremartra/Large-Language-Model-Notebooks-Course: Practical course about Large Language...

Practical course about Large Language Models. . Contribute to peremartra/Large-Language-Model-Notebooks-Course...

github.com

The first step I took with the original model in memory was to execute a small prompt and save the result. This allowed me to easily, visually, and quickly check

Open in app ↗

Medium

Search



not respected, the text produced left no doubt that the pruning process had a fundamental flaw.

The original prompt is: “Paris is the capital of.” Let’s look at the response from the original model and compare it to the one returned by my first, failed, pruning attempt.

Base Model:

“Paris is the capital of France and one of the most visited cities in the world. It is a city of art, culture, fashion, and gastronomy. The city has a rich history and is home to many famous landmarks, including the E.”

Incorrect model with only 20% pruning:

“Paris is the capital of of France. This is the the the the main the area of. This is the the the the the the the the the the the the the the the the city of the the France of the of the of the of.”

It’s clear that something didn’t work in that first attempt. It might seem trivial, but an empirical check like this can save you quite a few hours.

Implementation Details

Let’s start by looking at the function responsible for calculating the importance of the neurons, which will ultimately decide which neurons remain in the model and which ones are removed.

```
def compute_neuron_pair_importance(gate_weight, up_weight):  
    """  
    compute neuron pair importance scores (Maximum Absolute Weight)  
    Args:  
    - gate_weight: Weight matrix from the gate_proj layer.  
    - up_weight: Weight matrix from the up_weight layer.  
    Returns:  
    - importance_scores: Importance scores for each neuron pair.  
    """  
    gate_max_abs = torch.max(gate_weight, dim=1).values + torch.abs(torch.min(g  
    up_max_abs = torch.max(up_weight, dim=1).values + torch.abs(torch.min(up_we  
    importance_scores = gate_max_abs + up_max_abs  
    return importance_scores
```

The function receives the weights of a *gate_proj* layer and an *up_proj* layer, which, as I’ve explained, work in pairs. Therefore, the importance of the neurons must be

calculated jointly.

The calculation is very straightforward: it computes the absolute value of the weights for each neuron. Both positive and negative values are considered because, in theory, neurons with the most extreme values have a greater impact on the model's output by significantly altering the values passing through them.

Here, I must thank [MariusZ Kurman](#) for their contribution in incorporating the minimum values into the calculation. While the method worked correctly without them, their inclusion has improved the results.

The importance is calculated separately for each layer, but the function returns the combined value.

```
def prune_neuron_pairs(mlp, prune_percent):
    """
    Reduces the dimensions of the **gate_proj**, **up_proj**, **down_proj**
    layers removing the least important neurons.
    Args:
    - mlp: Layers to prune.
    - prune_percent: Percentage of neurons to prune.
    Returns:
    - new_gate_proj, new_up_proj, new_down_proj: New pruned layers.
    - k: New intermediate size.
    """
    # Extract weights from MLP layers
    gate_weight = mlp.gate_proj.weight.data.float()
    up_weight = mlp.up_proj.weight.data.float()

    # Compute importance scores
    importance_scores = compute_neuron_pair_importance(gate_weight, up_weight)
    original_intermediate_size = gate_weight.size(0)

    # Calculate neurons to keep
    num_neuron_pairs_to_prune = min(int(prune_percent * original_intermediate_size),
                                     original_intermediate_size - 1)
    k = original_intermediate_size - num_neuron_pairs_to_prune

    # Validation check
    if k <= 0:
        raise ValueError(f"Invalid number of neuron pairs to keep: {k}")

    # Select neurons to keep
    _, indices_to_keep = torch.topk(importance_scores, k, largest=True, sorted=False)
    indices_to_keep = indices_to_keep.sort().values

    # Create and populate new layers
```



```

new_gate_proj = nn.Linear(mlp.gate_proj.in_features, k, bias=False).to(device)
new_up_proj = nn.Linear(mlp.up_proj.in_features, k, bias=False).to(device)
new_down_proj = nn.Linear(k, mlp.down_proj.out_features, bias=False).to(device)

# Copy selected weights
new_gate_proj.weight.data = mlp.gate_proj.weight.data[indices_to_keep, :]
new_up_proj.weight.data = mlp.up_proj.weight.data[indices_to_keep, :]
new_down_proj.weight.data = mlp.down_proj.weight.data[:, indices_to_keep]

return new_gate_proj, new_up_proj, new_down_proj, k

```

This function creates new, smaller layers while preserving the most important neurons. The process involves:

- Extracting the current weights:

```

# Extract weights from MLP layers
gate_weight = mlp.gate_proj.weight.data.float()
up_weight = mlp.up_proj.weight.data.float()

```

- Computing importance scores for neuron pairs:

```

# Compute importance scores
importance_scores = compute_neuron_pair_importance(gate_weight, up_weight)
original_intermediate_size = gate_weight.size(0)

```

A tensor is obtained that contains the importance scores calculated for each neuron. These scores reflect each neuron's contribution to the final output, indicating which ones should be kept.

- Determining how many neurons to keep:

```

# Calculate neurons to keep
num_neuron_pairs_to_prune = min(int(prune_percent * original_intermediate_s

```

```
original_intermediate_size - 1)
k = original_intermediate_size - num_neuron_pairs_to_prune
```

The total number of neurons to keep is calculated using the pruning percentage provided as a parameter and the original size of the layers.

- Selecting the most important neurons:

```
# Select neurons to keep
_, indices_to_keep = torch.topk(importance_scores, k, largest=True, sorted=False)
indices_to_keep = indices_to_keep.sort().values
```

Torch is used to retrieve the neurons with the highest importance scores, while also sorting them from most to least important. Since torch returns the data in descending order, the sort method is used to rearrange them in ascending order, which is what we need.

- Creating new, smaller layers:

```
# Create and populate new layers
new_gate_proj = nn.Linear(mlp.gate_proj.in_features, k, bias=False).to(device)
new_up_proj = nn.Linear(mlp.up_proj.in_features, k, bias=False).to(device)
new_down_proj = nn.Linear(k, mlp.down_proj.out_features, bias=False).to(device)
```

Three new layers are created with dimensions adjusted based on the selected indices. In *new_gate_proj* and *new_up_proj*, the input dimensions are preserved while the output dimensions are reduced. Conversely, in *new_down_proj*, the input dimensions are adjusted while the output dimensions remain unchanged.

- Copying selected weights to the new layers:

```
#copy weights to the new layers.
new_gate_proj.weight.data = mlp.gate_proj.weight.data[indices_to_keep, :]
```

```
new_up_proj.weight.data = mlp.up_proj.weight.data[indices_to_keep, :]  
new_down_proj.weight.data = mlp.down_proj.weight.data[:, indices_to_keep]
```

The relevant weights are transferred from the original layers to the new ones, ensuring that only the weights corresponding to the selected neurons are retained.

Now, let's look at the function responsible for iterating over all the layers and constructing the modified model.

```
def update_model(model, prune_percent):  
    """  
    Modifies each MLP layer in the model to retain only the most  
    important neurons.  
    Args:  
    - model: Model to prune.  
    - prune_percent: Percentage of neurons to prune.  
    Returns:  
    - model: New pruned model.  
    """  
    new_intermediate_size = None  
  
    for idx, layer in enumerate(model.model.layers):  
        mlp = layer.mlp  
        new_gate_proj, new_up_proj, new_down_proj, new_size = prune_neuron_pair(  
            mlp, prune_percent)  
  
        mlp.gate_proj = new_gate_proj  
        mlp.up_proj = new_up_proj  
        mlp.down_proj = new_down_proj  
  
        if new_intermediate_size is None:  
            new_intermediate_size = new_size  
  
    model.config.intermediate_size = new_intermediate_size  
    return model
```

This function iterates through each layer of the model, applying the pruning process and updating the model's configuration to reflect the new architecture.

If the config file is not updated, the model cannot be used after being saved, whether on Hugging Face or locally. Many libraries, such as Hugging Face's Transformers, rely on *model.config* to interpret the model's architecture. If the

configuration does not match the actual structure, operations like fine-tuning or inference performed through these libraries may fail.

Results Analysis.

With this code, I've created several models, which are available on the Hugging Face Hub.

These include:

- Three models derived from Llama-3.2-1b, with 20%, 40%, and 60% of the neurons in the MLP layers pruned.
- One model based on Gemma-2-2B, pruned by 40%.

You can download these models and, in addition to using them, study their architecture and how it has changed compared to the original models they are based on.

Let's analyze the changes in the architecture after applying 20% pruning to the Llama3.2-1b model.

```
LlamaForCausalLM(  
  (model): LlamaModel(  
    (embed_tokens): Embedding(128256, 2048)  
    (layers): ModuleList(  
      (0-15): 16 x LlamaDecoderLayer(  
        (self_attn): LlamaSdpaAttention(  
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)  
          (k_proj): Linear(in_features=2048, out_features=512, bias=False)  
          (v_proj): Linear(in_features=2048, out_features=512, bias=False)  
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)  
          (rotary_emb): LlamaRotaryEmbedding()  
        )  
        (mlp): LlamaMLP(  
          (gate_proj): Linear(in_features=2048, out_features=6554, bias=False)  
          (up_proj): Linear(in_features=2048, out_features=6554, bias=False)  
          (down_proj): Linear(in_features=6554, out_features=2048, bias=False)  
          (act_fn): SiLU()  
        )  
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)  
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)  
      )  
    )  
    (norm): LlamaRMSNorm((2048,), eps=1e-05)  
    (rotary_emb): LlamaRotaryEmbedding()
```

```
)  
(lm_head): Linear(in_features=2048, out_features=128256, bias=False)  
)
```

The structure of the model remains unchanged except for the size of the intermediate layers in the MLP blocks. As you can see, the *gate_proj* and *up_proj* layers have been reduced from 8192 features to 6554, and the *down_proj* layer has undergone the same change, but in its input features.

This change is fully aligned with what the code does: modifying these layers while preserving the neurons that are most critical for the model's performance. If we remove 20% of 8192, we get 6553.6, confirming that the correct percentage of neurons has been pruned.

Empirical prompt testing.

Now, let's see how the pruned model performed with the test prompt:

Paris is the capital of France. It is also one of the most beautiful cities in the world. There is so much to see and do in Paris that it is impossible to cover it all in one day. However, there are some things you

The response isn't identical to the one from the original model, but it maintains coherence. This suggests that the model retains much of its capabilities, and more importantly, it could potentially recover any losses through **knowledge distillation** or **fine-tuning**.

EleutherAI / lm-evaluation.

Beyond this empirical check, I've also evaluated the model using some of the most common benchmarks. Let's analyze how different degrees of pruning affect the model's performance.

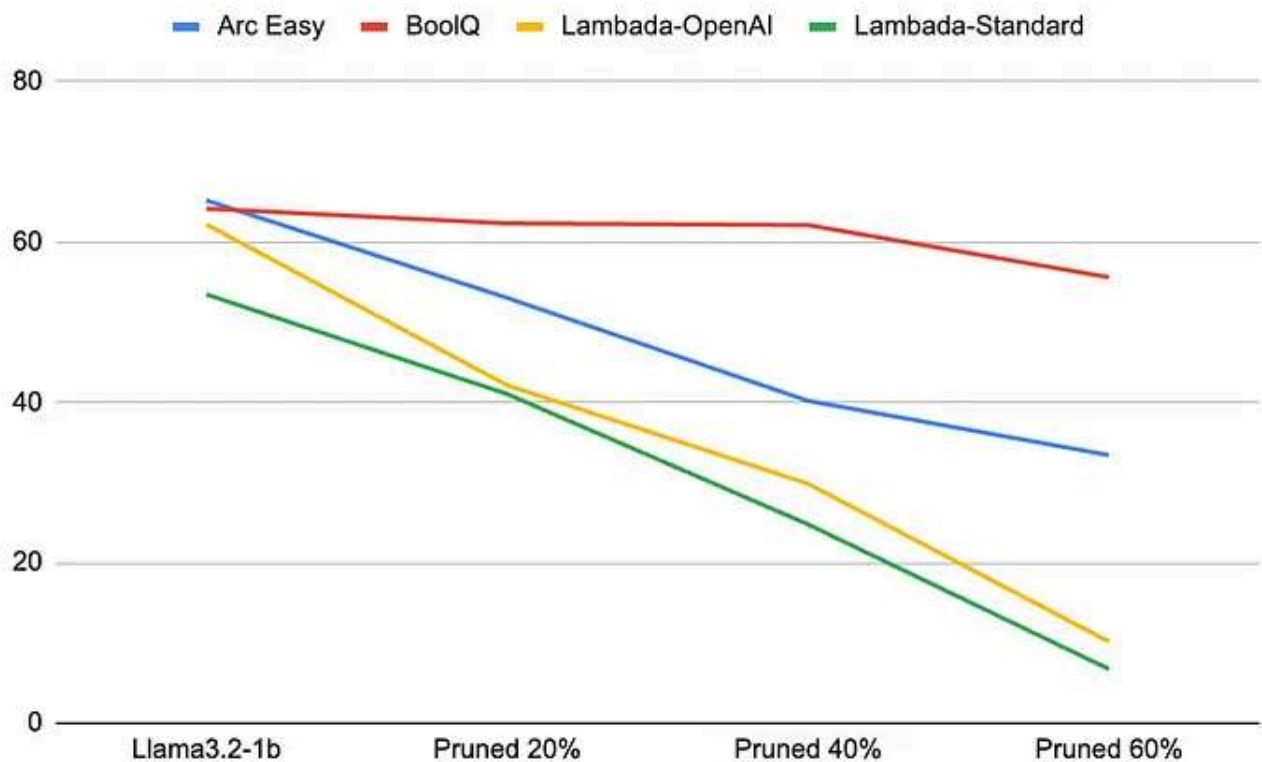


Image by Author. Models performance on different metrics

As we can see, the effect of pruning has been somewhat asymmetrical. The tasks evaluated by the BoolQ test haven't experienced significant degradation, only about a 2% drop for a model that lost 40% of the neurons in the MLP layers.

In contrast, the impact on the Lambada test has been remarkable, with a drop in accuracy of over 50%.

This indicates that the model retains much of its comprehension ability but struggles with tests requiring more open-ended generation.

BoolQ simply presents the model with a text and a question to be answered with Yes/No. It's a test focused on measuring the model's ability to understand relationships within the input text.

Lambada, on the other hand, asks the model to guess the last word of a paragraph, a complex task where the final word tests the model's capability in complex language modeling.

Hugging Face Open LLM Leaderboard.

The results of the model pruned to 20% on the Hugging Face [Open LLM Leaderboard](#) are perhaps even more surprising, as it outperforms both its base model and the widely [used TinyLlama-1.1B-v1.1](#).

In this graph we can see the results of both models.

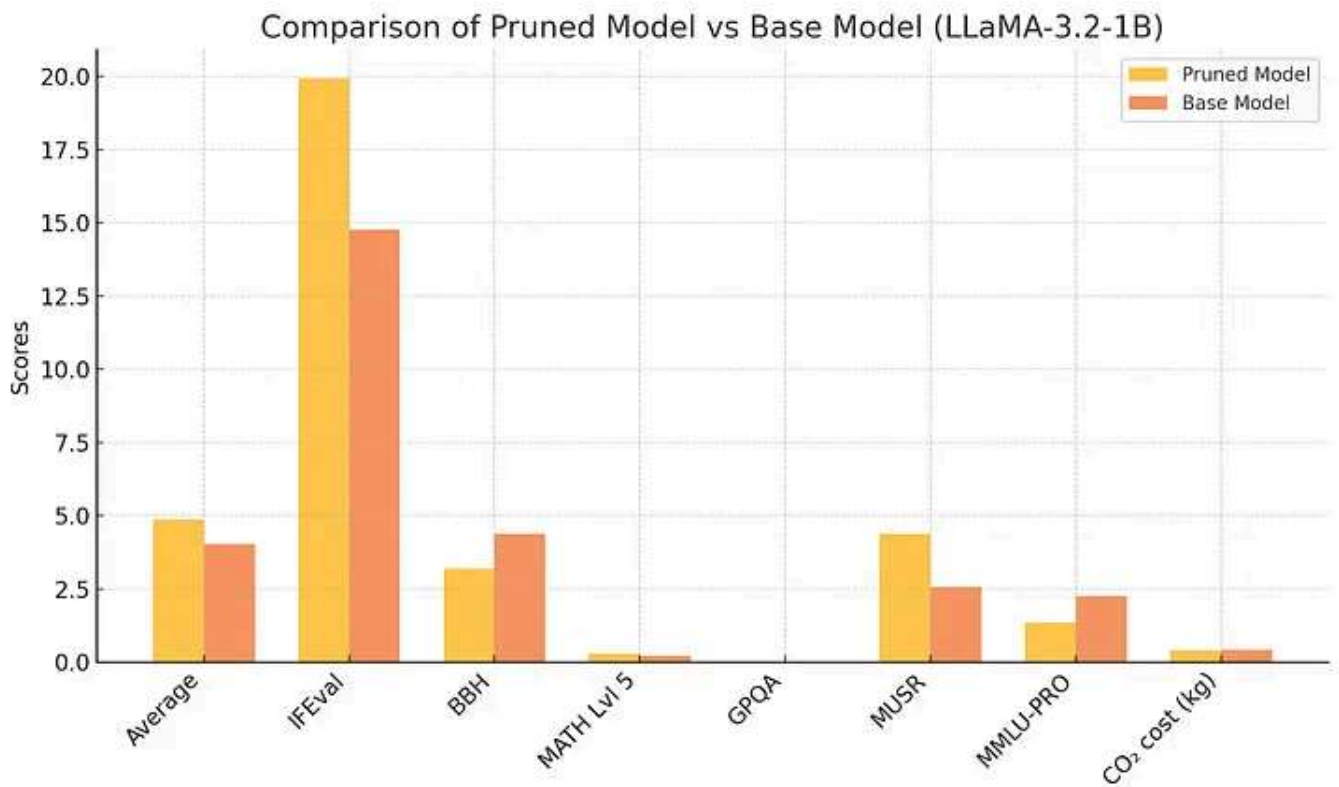


Image Generated by Author using GPT.

From studying this graph, we could draw the following conclusions: The **pruned model outperforms the base model on average (4.86 vs. 4.03)**. This suggests that the pruning process has effectively retained or enhanced performance in key areas while reducing redundancy.

Studying the results we can identify Strengths and Weaknesses of the pruned model.

Strengths:

- **IFEval:** Significant improvement (19.94 vs. 14.78) suggests that pruning either reduced overfitting or improved the model's ability to extract information efficiently.
- **MUSR:** Better performance (4.39 vs. 2.56) indicates that the pruned model handles tasks requiring reasoning over long contexts or narrative understanding better, possibly due to focused weights.

Weaknesses:

- **BBH:** Decline in reasoning under uncertainty (3.19 vs. 4.37) may indicate pruning reduced the model's capacity for handling ambiguous or multi-interpretation scenarios.
- **MMLU-PRO:** A drop in professional domain-specific tasks (1.36 vs. 2.26) could be due to the removal of weights crucial for retaining detailed knowledge in specific areas.

Energy Efficiency: The pruned model is slightly more energy-efficient (0.4 kg vs. 0.42 kg CO₂), aligning with the goal of reducing computational overhead while maintaining competitive performance.

A more complete study of the model's performance across different rankings would be needed, but these results suggest we have a promising model that could improve significantly with proper knowledge distillation or fine-tuning. Most importantly, these results align with the pruning procedure performed on the MLP layers.

Conclusions.

The pruning process for the models has been a success. This approach to handling GLU layers allows us to perform pruning while retaining a significant portion of the model's capabilities, thereby reducing its size and resource consumption considerably.

It's important to note that the test results were obtained with the pruned model *before* undergoing any capability recovery process, such as **knowledge distillation** or **fine-tuning**, which is typically done for models that have undergone pruning.

Future Work.

There are many pruning techniques worth exploring. Perhaps the most straightforward is **depth pruning**, which involves removing layers that contribute the least to the model's performance.

Another essential area of research would be to subject these pruned models to a **knowledge distillation** process and evaluate whether they retain the ability to learn new tasks. This could potentially bring their performance closer to that of the base model, particularly in the benchmarks where the pruned model showed the most significant losses.

The development of lighter, more efficient models remains an attractive field, particularly for companies seeking to deploy LLM capabilities without extensive

infrastructure requirements. This work provides a foundation for further research in making these powerful models more accessible and deployable.

This article is part of a full course about [Large Language Models](#), available at [Github](#). To stay updated on new articles, please consider following the repository or starring it. This way, you'll receive notifications whenever new content is added.

I'm the author of the book "[Large Language Models Projects: Apply and Implement Strategies for Large Language Models](#)" published by Apress.

I write about Generative AI, Deep Learning and TensorFlow regularly. **Consider following me on Medium to get updates about new articles.** And, of course, You are welcome to connect with me on [LinkedIn](#).

Large Language Models

Llama 3

Pruning

Small Language Model

Hands On Tutorials



Follow

Published in TDS Archive

805K Followers · Last published Feb 3, 2025

An archive of data science, data analytics, data engineering, machine learning, and artificial intelligence writing from the former Towards Data Science Medium publication.



Follow

Written by **Pere Martra** 

635 Followers · 144 Following

AI Engineer. Authoring a course about Large Language Models Applications & Techniques. I write, mainly, about LLMs, DeepLearning and TensorFlow.

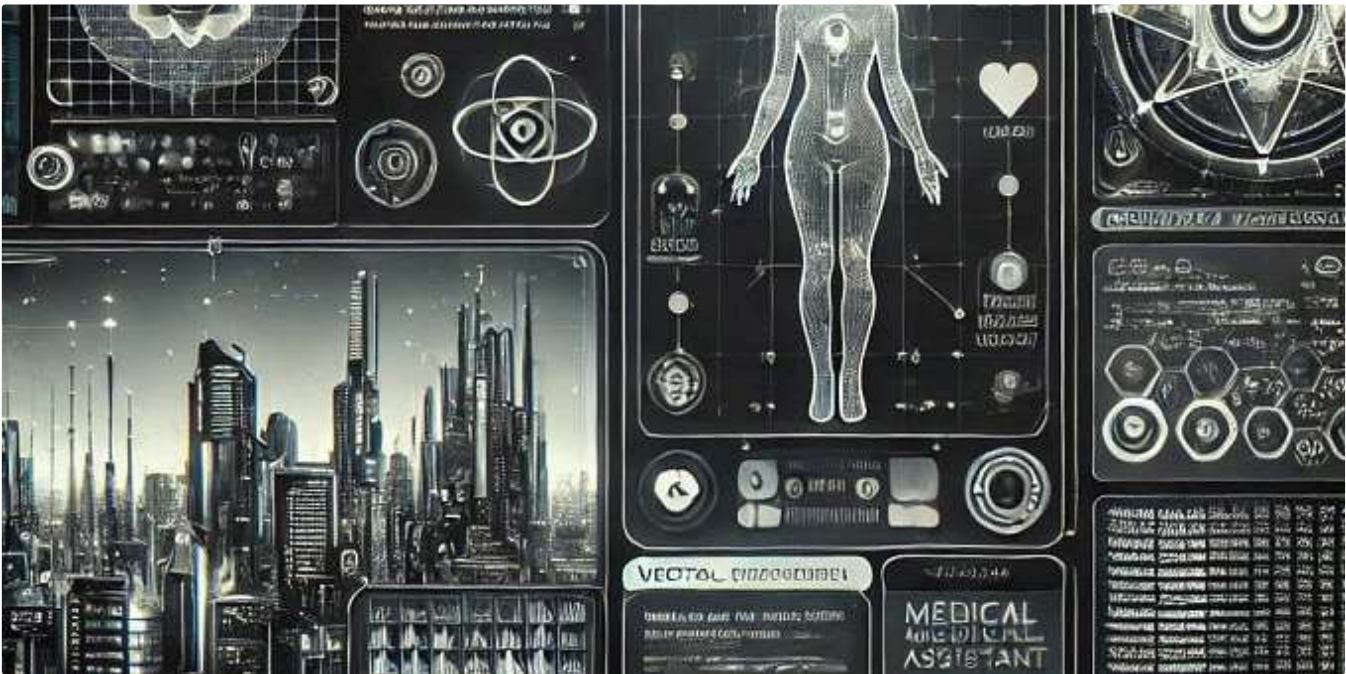
No responses yet



What are your thoughts?

Respond

More from Pere Martra and TDS Archive



 In Towards AI by Pere Martra 

How to Create a Medical Agent / RAG System.

Learn how to build a ReAct agent powered by a RAG system using LangChain, ChromaDB, and OpenAI, with a user-friendly Gradio interface.



tds In TDS Archive by Marina Wyss - Gratitude Driven

How to Pick Between Data Science, Data Analytics, Data Engineering, ML Engineering, and SW...

Make the right choice for YOU



tds In TDS Archive by Eric Broda

Agentic Mesh: Towards Enterprise-Grade Agents

Enterprise-grade capabilities will make or break the future of agent adoption. Without them, agents adoption will collapse.

★ Jan 21 🖱 349 💬 8

🔖+ ⋮



 In Towards AI by Pere Martra 

Evaluating a RAG Solution with Giskard

Giskard is an open-source project dedicated to providing a framework for the evaluation of different machine-learning models. Here, you...

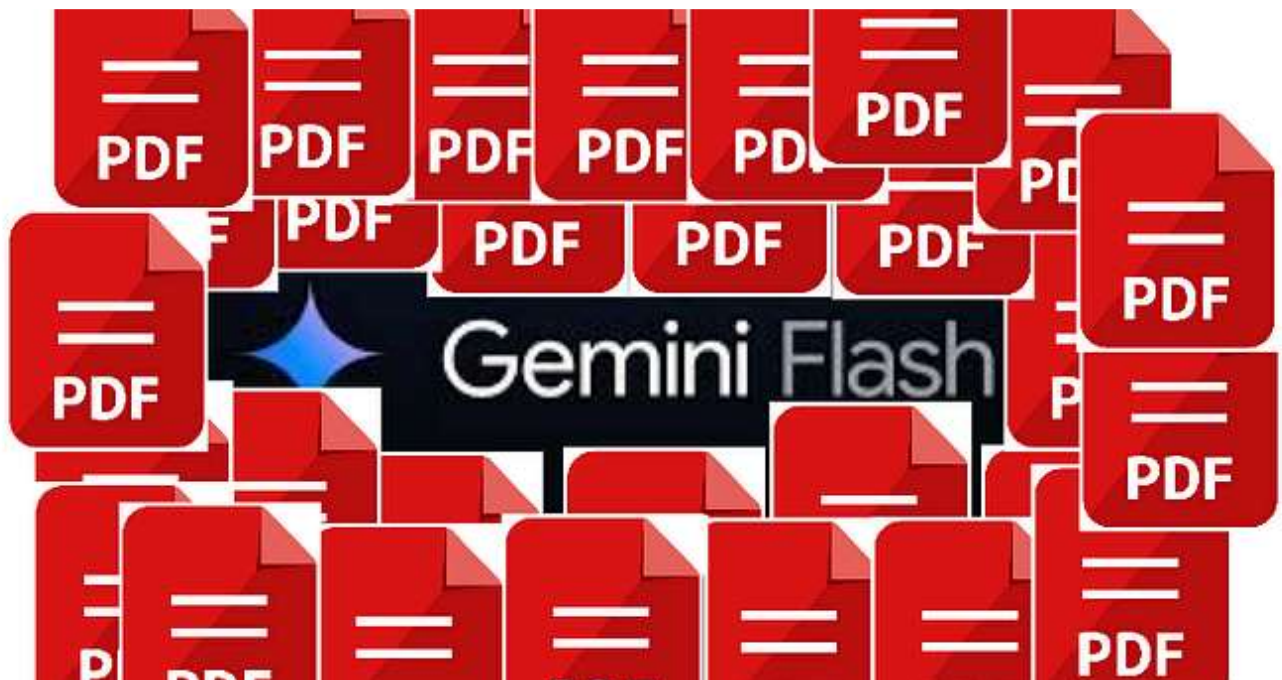
★ Oct 7, 2024 🖱 76

🔖+ ⋮

See all from Pere Martra

See all from TDS Archive

Recommended from Medium



 In AI Advances by Michael Ryaboy

10x Cheaper PDF Processing: Ingesting and RAG on Millions of Documents with Gemini 2.0 Flash

Picture this: you start by converting every PDF page into images, then send them off for OCR, only to wrestle the raw text into workable...

6d ago  387  6



 In Towards AI by Alex Punnen

Explaining Transformers as Simple as Possible through a Small Language Model

And understanding Vector Transformations and Vectorizations

★ 5d ago 🖱 591 💬 13



Lists



Natural Language Processing

1946 stories · 1597 saves



AI Regulation

6 stories · 691 saves



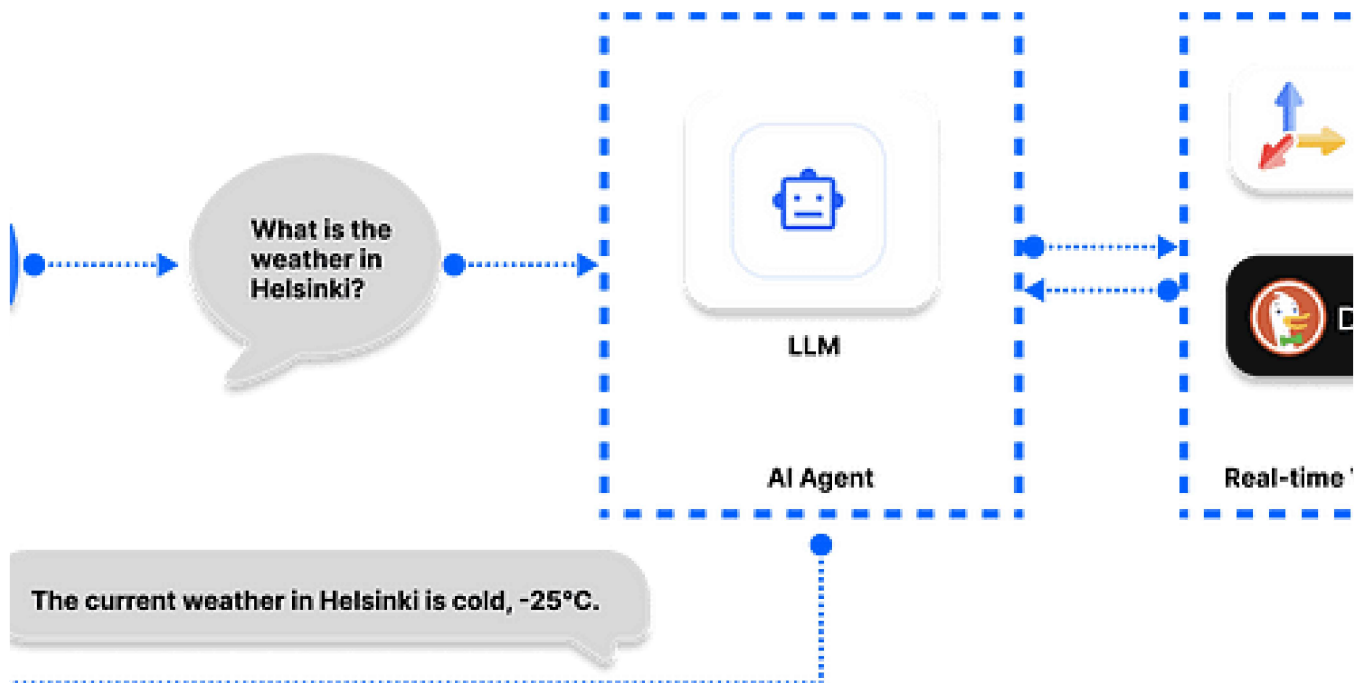
Stories to Help You Grow as a Software Developer

19 stories · 1603 saves



Generative AI Recommended Reading

52 stories · 1658 saves



 Amos Gyamfi

The Best Pre-Built Toolkits for AI Agents

Python and TypeScript-based AI agent frameworks, CrewAI, LangChain, Agno, and Vercel AI SDK allow developers to build AI applications with...

Feb 10  349  2

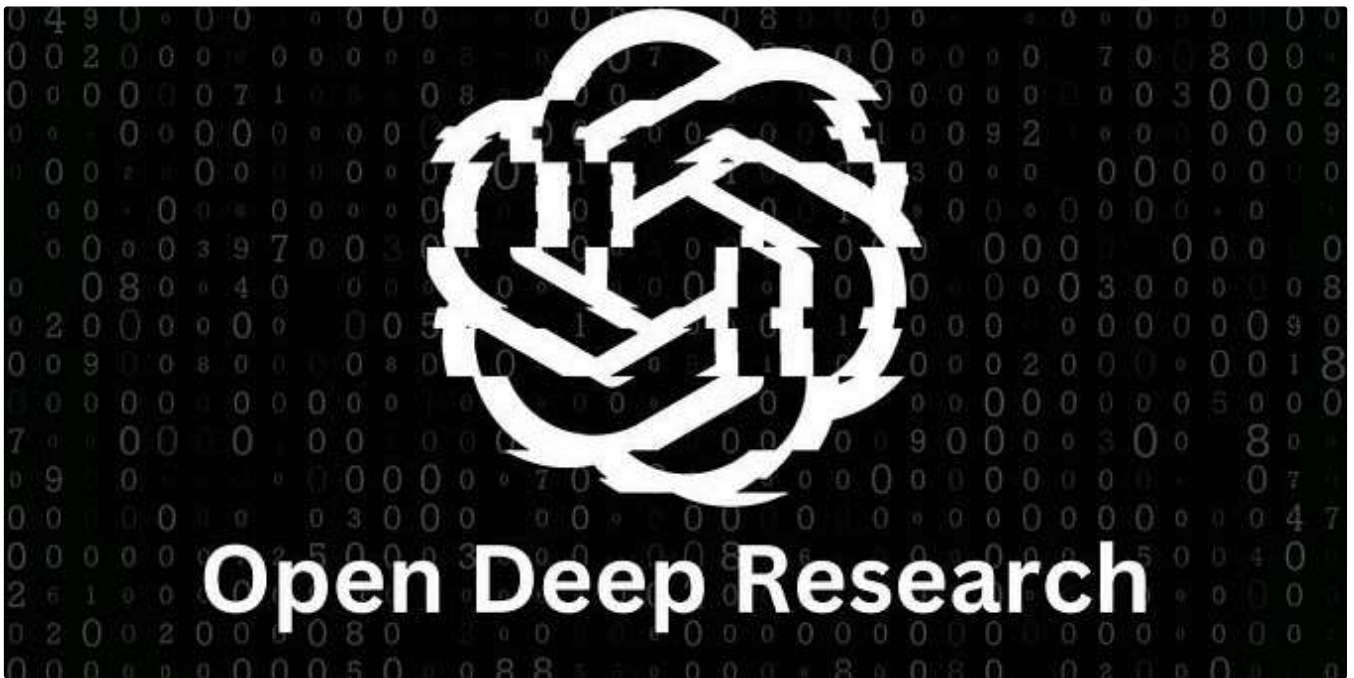
 



 Shaw Talebi

How to Train LLMs to “Think” (o1 & DeepSeek-R1)

Advanced reasoning models explained



📺 In Generative AI by Jim Clyde Monge 🏠

How To Use “Open Deep-Research”: An Open Source Alternative To OpenAI’s \$200 Deep Research AI Agent

A step-by-step guide on how to setup and use Open Deep Research on your local machine.



📺 In Level Up Coding by Cristian Leo

Transformer-Squared: Stop Finetuning LLMs

The architecture behind self-adaptive LLMs, the math and code behind Transformer-Squared, and Single Value Decomposition.

 Feb 10  453  4

See more recommendations